

Detailing Service Booking System

Root Cause Analysis & Comprehensive Solution Blueprint

Author: Antigravity AI Engineering

Project: Mentor Engineers Automotive Workshop
(Customized OS)

Date: September 22, 2026

Target File: `index.ts` (ai-chat-mentor-engineeros)

Executive Summary

During testing of the live production WhatsApp AI booking system for Mentor Engineers, two major issues were observed when selecting the **Detailing Service**:

- Wrong Year / Stale Date Suggestion:** The AI assistant suggested booking dates in the year **2024** instead of upcoming valid dates in **2026**.
- Missing / Unenforced Default Time Slot (8:30 AM):** Detailing is a full-day operation requiring vehicle intake strictly at **8:30 AM**. However, the bot prompted customers generically for an open "Preferred Time Slot", allowing arbitrary arrival hours.

This report provides the exhaustive technical root causes, mathematical date-engine flaws, rule adherence verifications, and a step-by-step code and prompt engineering solution blueprint.

1. Deep Root Cause Analysis

Issue 1: Why did the AI suggest a 2024 date?

Root Cause 1.1: LLM Date Calculation vs. Training Data Cutoff

In git commit history (`26ff065` and `7cddcf6`), the edge function did **not** pre-compute dates in TypeScript. Instead, it sent instructions to the LLM (`google/gemini-3-flash-preview`):

```
When the customer wants to book a detailing service, you MUST explicitly suggest the NEXT 3
AVAILABLE DATES for them to choose from.
To find the next 3 available dates:
1. Start from Tomorrow.
2. Check if the date is a Tuesday, a Poya day, a bridge holiday, or already booked...
```

Why this fails: Large Language Models are generative probabilistic text models, not deterministic calendar engines. When an LLM with training data centered around 2023–2024 is instructed to calculate future dates, its attention heads frequently latch onto statistical date tokens from its pre-training weights (specifically 2024). While a recent commit attempted to fix this, **the live production server (178.104.127.220) was still running the older edge function build.**

Root Cause 1.2: Fragile JavaScript Date & Timezone Math

The date logic in the edge function was fundamentally flawed:

```
const getSLTime = (date = new Date()) => new Date(date.getTime() + (5.5 * 60 * 60 * 1000));
```

Issue 2: Why did the default 8:30 AM time slot fail?

Root Cause 2.1: Prompt Conflict in Workflow Rules

In the general conversation workflow, the prompt explicitly asked the AI to collect: *"Preferred Appointment Date & Time Slot"*. Because the word **"Time Slot"** was in the instruction, the AI naturally asked the customer what time they preferred, leading to arbitrary responses (e.g., "10:30 AM").

Root Cause 2.2: Late Notification of 8:30 AM Arrival

The instruction stating, *"instruct them to arrive at the workshop at 8:30 AM,"* only appeared at line 281. This comes **too late** in the dialogue. The bot asks for a slot, the customer provides one, and the bot either accepts it or creates confusion by changing it at the final step.

Root Cause 2.3: Lack of Backend Fallback & Enforcement

In the order parsing block, there was no backend guard verifying that if `serviceCategory === 'detailing'`, the `customFields.booking_time` **must be hard-locked to "08:30 AM"**. It blindly accepted whatever string the LLM passed.

2. Business Rules & Operational Specifications

Mentor Engineers adheres to strict workshop bay constraints for vehicle detailing. The following rules must be algorithmically enforced before the LLM generates a response.

Parameter	Specification	Enforcement Mechanism
Timezone	Asia/Colombo (UTC +05:30)	Standard Intl.DateTimeFormat across manipulations
Bay Capacity	Strictly 1 Vehicle Per Day	Checked against active DB orders
Weekly Closure	Every Tuesday is strictly closed	Day of week 2 check in engine
Religious Holidays	All Poya Days are strictly closed	5-Year Verified Sri Lankan Poya Calendar
Bridge Holidays	Sandwiched between Tuesday and Poya	Pre-day and Next-day neighbor inspection
Vehicle Arrival	Strictly 8:30 AM	Locked in <ORDER_JSON> & DB
Suggestions	Next 3 consecutive valid dates	Pre-computed in deterministic TypeScript

Deterministic Booking Flow Logic

1. Retrieve Current Date in Asia/Colombo.
2. Loop next consecutive days. For each day, check: Is it Tuesday? Is it Poya? Is it a Bridge Holiday? Is it already booked?
3. If valid, add to list. Repeat until 3 dates are found.
4. Inject EXACT 3 dates and strict 8:30 AM rule into AI Context.
5. Bot presents options. If user picks valid date, generate JSON forcing 08:30 AM .

3. Comprehensive Technical Solution Blueprint

3.1. Robust Sri Lanka Timezone & Date Math

Instead of fragile manual millisecond addition and `.toISOString()`, we use deterministic UTC string arithmetic calibrated to `Asia/Colombo`:

```
// 1. Get today's date strictly formatted in Asia/Colombo timezone
function getSriLankaDateString(d = new Date()): string {
  const formatter = new Intl.DateTimeFormat("en-CA", {
    timeZone: "Asia/Colombo",
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  });
  return formatter.format(d); // Outputs guaranteed "YYYY-MM-DD"
}

// 2. Add days cleanly using UTC timestamps to prevent daylight drift
function addDays(dateStr: string, days: number): string {
  const [y, m, d] = dateStr.split("-").map(Number);
  const date = new Date(Date.UTC(y, m - 1, d + days, 12, 0, 0));
  return date.toISOString().split("T")[0];
}

// 3. Get day of week (0=Sun, 1=Mon, 2=Tue, etc.)
function getDayOfWeek(dateStr: string): number {
  const [y, m, d] = dateStr.split("-").map(Number);
  const date = new Date(Date.UTC(y, m - 1, d, 12, 0, 0));
  return date.getUTCDay();
}
```

3.2. Detailing Availability Engine

```
const poyaDays = [
  "2025-01-13", "2025-02-12", "2025-03-13", /* ... full array ... */ "2026-09-25"
];

const isDateAvailable = (dateStr: string, bookedList: string[]): boolean => {
  const day = getDayOfWeek(dateStr);
  if (day === 2) return false; // Tuesday
  if (poyaDays.includes(dateStr)) return false; // Poya
  if (bookedList.includes(dateStr)) return false; // Booked

  // Bridge holiday check
  const prevDateStr = addDays(dateStr, -1);
  const nextDateStr = addDays(dateStr, 1);
  const prevIsTue = getDayOfWeek(prevDateStr) === 2;
  const prevIsPoya = poyaDays.includes(prevDateStr);
  const nextIsTue = getDayOfWeek(nextDateStr) === 2;
  const nextIsPoya = poyaDays.includes(nextDateStr);

  if ((prevIsTue && nextIsPoya) || (prevIsPoya && nextIsTue)) return false;
  return true;
}
```

3.3. Prompt Engineering Overhaul

The calculated dates must be explicitly injected into the prompt, overriding any internal LLM calendar assumptions.

Context Injection (`holidayRulesContext`):

```
[DETAILING_SERVICE_AVAILABILITY]
Current Date in Sri Lanka: ${todaySL}
Booked Dates (Max 1 Car/Day): ${bookedDates.length > 0 ? bookedDates.join(", ") : "None currently booked"}
Tuesdays are ALWAYS closed for detailing.
Poya days are ALWAYS closed.
Bridge days between Tuesday and Poya are closed.

THE NEXT 3 AVAILABLE DATES FOR DETAILING ARE:
${next3AvailableDates.map((d, i) => ` ${i + 1}. ${d}`)}.join("\n")}

CRITICAL OPERATIONAL RULES FOR DETAILING:
1. STRICT TIME SLOT: Detailing is a comprehensive full-day service taking 8+ hours. Vehicle drop-off time is STRICTLY 8:30 AM.
2. DO NOT ASK CUSTOMERS FOR A TIME SLOT! Customers NEVER choose a time slot for detailing.
3. When suggesting dates, you MUST state the 3 dates AND inform the customer that arrival is at 8:30 AM:
   - English: "Sir / Madam, our next available dates for Detailing are: [Date 1], [Date 2], or [Date 3]. As detailing is a full-day process, vehicle drop-off is strictly at 8:30 AM. Which date would you prefer?"
4. If the customer proposes an unlisted, booked, Tuesday, or Poya date, politely reject it and re-iterate the 3 available dates.
5. In <ORDER_JSON>, always set "booking_time": "08:30 AM" for Detailing.
```

Workflow Rule Fix (Lines 443 & 496):

Update step 5 in the standard flow from:

17 Preferred Appointment Date & Time Slot

To:

17 Preferred Appointment Date (For Detailing: explicitly present the 3 dates and announce the strict 8:30 AM arrival time. For general Wash/Mechanical: ask for Preferred Date & Time Slot)

3.4. Backend Safety Guard (Order Ingestion)

In the `<ORDER_JSON>` database storage block (Line 668), enforce the time at the code level:

```
let bDate = orderData.booking_date || orderData.date;
let bTime = orderData.booking_time || orderData.time;

// Force 8:30 AM default for detailing
if (serviceCategory === "detailing") {
  bTime = "08:30 AM";
}
```

4. Verification & Testing Matrix

Before closing the issue, the following scenarios must be validated on the production endpoint.

Test Case	Scenario	Expected Behavior
TC-1: Current Year	User asks: "I want to book detailing"	Bot replies with current year (e.g., 2026-09-23). Never 2024.
TC-2: Tuesday Skip	Next date is a Tuesday (e.g. 2026-09-29)	Tuesday is skipped by date engine. Not suggested.
TC-3: Poya Skip	Next date is a Poya day (e.g. 2026-09-25)	Date is automatically skipped. Not suggested.
TC-4: Max 1 Car/Day	Order already booked for 2026-09-24	2026-09-24 marked booked; engine skips to next valid day.
TC-5: Fixed 8:30 AM	User asks for detailing dates	Bot presents 3 dates and immediately instructs arrival at 8:30 AM. Does NOT ask for time.
TC-6: Time Rejection	User says: "Can I come at 2:00 PM?"	Bot explains full-day service requires 8:30 drop-off.
TC-7: Order Storage	Order confirmation by customer	<ORDER_JSON> created with booking_time: "08:30 AM" .
TC-8: UI Calendar	User views Admin Calendar	Order displays with 8:30 AM - 5:30 PM block.

5. Deployment & Execution Procedure

- Apply Code Edits:** Update `supabase/functions/ai-chat-mentor-engineeros/index.ts` with the new timezone engine, prompt guidelines, and backend fallback.
- Git Commit & Push:** Commit the changes to `master` branch to track version control.
- Production Edge Function Sync:** Deploy the updated function directly to the live server (`178.104.127.220`) into Docker volume `/root/supabase/supabase/docker/volumes/functions/ai-chat-mentor-engineeros/index.ts` .
- Restart Edge Runtime:** Run `docker restart supabase-edge-functions` to load the fresh function code into memory.
- Live WhatsApp E2E Test:** Send a test message on WhatsApp selecting "3 – Detailing" to verify the exact 3 dates and the 8:30 AM arrival instruction are functioning correctly.